The CE approach, first proposed in the paper by Bertsekas and Castañon [BeC99], has an important property: *it maintains the Newton step character of the approximation in value space scheme*. In particular, the function $J_{\tilde{\mu}}$ of the ℓ-step lookahead policy $\tilde{\mu}$ obtained is generated by a Newton step, applied to the function obtained by the last $\ell-1$ minimization steps (modified by CE, and applied to the terminal cost function approximation); see the monograph [Ber20a] for a discussion. Thus the benefit of the fast convergence of Newton's method is restored. In fact based on insights derived from this Newton step interpretation, it appears that the performance penalty for the CE approximation is typically small. At the same time the ℓ-step lookahead minimization involves only one stochastic step, the first one, and hence potentially a much "thinner" lookahead graph, than the ℓ-step minimization that does not involve any CE-type approximations; see Fig. 2.7.1. Moreover, the ideas of tree pruning and iterative deepening, which we have discussed in Section 2.4 for deterministic multistep lookahead, come into play when the CE approximation is used.

Certainty Equivalence Approximations in Rollout

Certainty equivalence ideas can also be used in the context of rollout with a base policy. The cost function $J_\mu\big(f(x,u,w)\big)$ of the base policy μ in the rollout control calculation

$$\tilde{\mu}(x) \in \arg\min_{u\in U(x)} E_w\Big\{g(x,u,w) + \alpha J_\mu\big(f(x,u,w)\big)\Big\}, \quad \text{for all } x, \qquad (2.76)$$

[cf. Eq. (2.75)], can be approximated by the cost function

$$\bar{J}_\mu\big(f(x,u,w)\big)$$

of μ for a deterministic problem where the stochastic disturbances w are replaced by "typical" deterministic quantities $\overline{w}$. If w is a continuous random variable, then it is natural to use $\overline{w} = E\{w\}$. If w is a discrete random variable, then one possibility is to use as $\overline{w}$ the most probable value of w under the state-control pair $\big(x,\mu(x)\big)$. The resulting scheme can be viewed as an approximate rollout algorithm in the context of approximation in value space, but with exact minimization at the first step of lookahead [cf. Eq. (2.76)]. For this reason it still implements a Newton step for solving the Bellman equation and inherits the attendant cost improvement properties.

Another possibility, which applies to problems of finite horizon N, is to calculate the rollout control at state x_k by computing the most likely future disturbance trajectory $(w_{k+1},\ldots,w_{N-1})$ under the base policy; cf. Section 2.3.7. Similar ideas also apply to truncated rollout schemes that incorporate CE approximations.

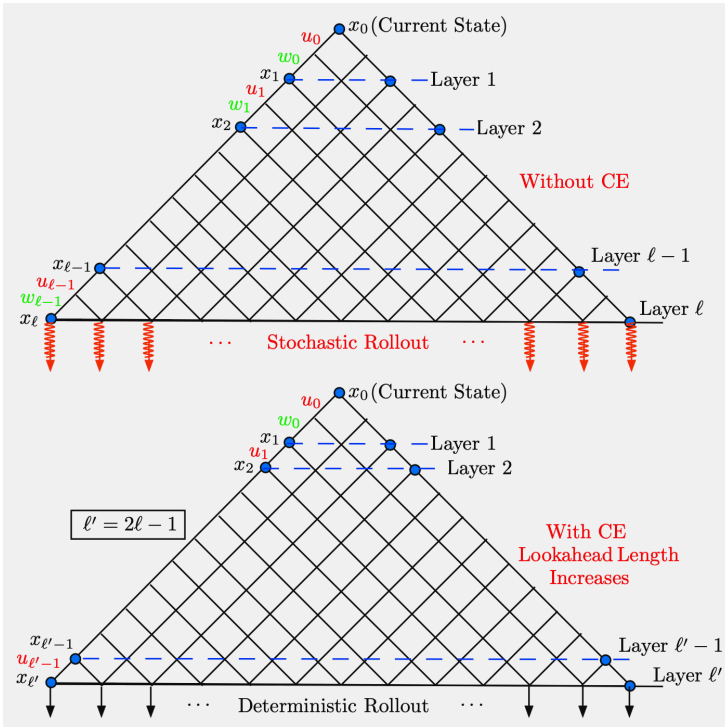

Figure 2.7.1 Illustration of the computational savings with the CE approximation, applied at the states after the first layer of states of the multistep lookahead tree. The figure on the top (or the bottom) illustrates the lookahead tree without (or with, respectively) CE. It can be seen that with CE, the lookahead tree grows much faster (the layers contain more states). In particular, the "height" of the ℓ-step lookahead graph without CE is the same as the "height" of a ℓ'-step lookahead graph with CE, where $\ell' = 2\ell - 1$. Moreover, with a number m of controls per state, and a number n of disturbances per state-control pair, the number of leaves of the ℓ-step lookahead tree is estimated as $O(mn\ell)$ without CE and $O\big(m(n+\ell)\big)$ with CE.

Example 2.7.3 (Markov Jump Parameter Problems)

Consider an N-stage horizon problem where the uncertainty in the system equation comes from a parameter θ_k that is perfectly observed, but evolves according to a Markov chain with transition probabilities

$$p_{ij}^k(x_k, u_k) = P(\theta_{k+1} = j \mid \theta_k = i, x_k, u_k), \qquad i, j = 1, \ldots, r,$$

where $1, \ldots, r$, are the possible values of the parameter. For example, in a maintenance-and-repair context, the system can be in one of two states, "good" and "bad", and the problem may be to schedule preventive maintenance of the system to maximize adherence to a certain production objective.

To capture the correlation between successive parameter values, we introduce an augmented state (x_k, θ_k), and policies $\mu_k(x_k, \theta_k)$ that depend on this augmented state. To this end, we must define a stochastic dynamic system that describes the evolution from the augmented state (x_k, θ_k) to the next augmented state (x_{k+1}, θ_{k+1}). We thus introduce a stochastic variable w_k that models the next value of the parameter, $\theta_{k+1} = w_k$, and changes according to the given Markov chain transition probabilities; see our discussion of state augmentation in Section 1.6.5. The augmented state evolves according to

$$x_{k+1} = f_k(x_k, u_k, \theta_k), \qquad \theta_{k+1} = w_k, \qquad k = 1, \ldots, N-1,$$

where w_k takes the values $i = 1, \ldots, r$ with probabilities $p^k_{i\theta_k}(x_k, u_k)$.

Suppose now that we want to apply rollout with a given base policy, which at state parameter pair (x_k, θ_k) applies control $\mu_k(x_k, \theta_k)$. To implement a CE approximation, we must calculate the base policy controls for the future time periods, $k+1, \ldots, N-1$, while assuming a deterministic trajectory of parameter values after θ_{k+1}. One possibility is to compute this trajectory as the most likely future trajectory $(w_{k+1}, \ldots, w_{N-1})$ under the base policy. This computation must be done for each of the possible pairs (x_{k+1}, θ_{k+1}), using the Markov chain probabilities that correspond to the state and control pairs that are generated by the base policy starting with (x_{k+1}, θ_{k+1}).

2.7.3 Simulation-Based Implementation of the Rollout Algorithm

A conceptually straightforward way to compute the rollout control at a given state x_k and time k is to consider each possible control $u_k \in U_k(x_k)$, and to generate a "large" number of simulated trajectories of the system starting from (x_k, u_k). Thus a simulated trajectory is obtained from

$$x_{i+1} = f_i\big(x_i, \mu_i(x_i), w_i\big), \qquad i = k+1, \ldots, N-1,$$

where $\{\mu_{k+1}, \ldots, \mu_{N-1}\}$ is the tail portion of the base policy, the starting state of the simulated trajectory is

$$x_{k+1} = f_k(x_k, u_k, w_k),$$

and the disturbance sequence $\{w_k, \ldots, w_{N-1}\}$ is obtained by random sampling. The costs of the trajectories corresponding to a pair (x_k, u_k) can be viewed as samples of the Q-factor

$$Q_{k,\pi}(x_k, u_k) = E\Big\{g_k(x_k, u_k, w_k) + J_{k+1,\pi}\big(f_k(x_k, u_k, w_k)\big)\Big\},$$

where $J_{k+1,\pi}$ is the cost-to-go function of the base policy, i.e., $J_{k+1,\pi}(x_{k+1})$ is the cost of using the base policy starting from x_{k+1}. For problems with a large number of stages, it is also common to truncate the rollout trajectories

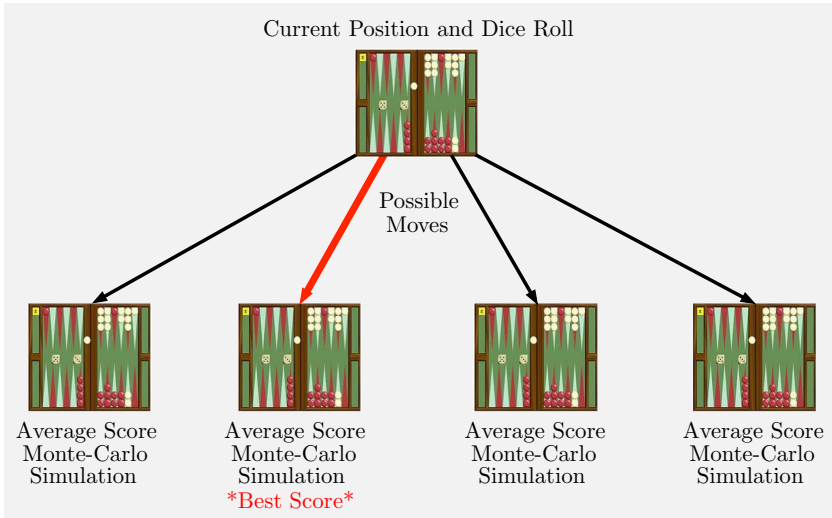

Figure 2.7.2 Illustration of rollout for backgammon. At a given position and roll of the dice, the set of all possible moves is generated, and the outcome of the game for each move is evaluated by "rolling out" (simulating to the end) many games using a suboptimal/heuristic backgammon player (the TD-Gammon player was used for this purpose in [TeG96]), and by Monte Carlo averaging the scores. The move that results in the best average score is selected for play.

and add a terminal cost function approximation as compensation for the resulting error.

By Monte Carlo averaging of the costs of the sample trajectories plus the terminal cost (if any), we obtain an approximation to the Q-factor $Q_{k,\pi}(x_k, u_k)$ for each $u_k \in U_k(x_k)$, denoted by $\tilde{Q}_{k,\pi}(x_k, u_k)$. We then compute the (approximate) rollout control $\tilde{\mu}_k(x_k)$ with the minimization

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \tilde{Q}_{k,\pi}(x_k, u_k). \tag{2.77}$$

Example 2.7.4 (Backgammon)

The first impressive application of rollout was given for the ancient two-player game of backgammon, in the paper by Tesauro and Galperin [TeG96]; see Fig. 2.7.2. They implemented a rollout algorithm, which attained a level of play that was better than all computer backgammon programs, and eventually better than the best humans. Tesauro had proposed earlier the use of one-step and two-step lookahead with lookahead cost function approximation provided by a neural network, resulting in a backgammon program called TD-Gammon [Tes89a], [Tes89b], [Tes92], [Tes94], [Tes95], [Tes02]. TD-Gammon was trained with an approximate policy iteration method, and was used as the base policy (for each of the two players) to simulate game trajectories.

The rollout algorithm also involved truncation of long game trajectories, using a terminal cost function approximation based on TD-Gammon's position evaluation. Game trajectories are of course random, since they involve the use of dice at each player's turn. Thus the scores of many trajectories have to be generated and averaged with the Monte Carlo method to assess the probability of a win from a given position.

An important issue to consider here is that backgammon is a two-player game and not an optimal control problem that involves a single decision maker. While there is a DP theory for sequential zero-sum games, this theory has not been covered in this book. Thus how are we to interpret rollout algorithms in the context of two-player games, with both players using some base policy? The answer is to view the game as a (one-player) optimal control problem, where one of the two players passively uses the base policy exclusively (TD-Gammon in the present example). The other player takes the role of the optimizer, and actively tries to improve on his base policy (TD-Gammon) by using rollout. Thus "policy improvement" in the context of the present example means that when playing against a TD-Gammon opponent, the rollout player achieves a better score on the average than if he/she were to play with the TD-Gammon strategy. In particular, the theory does not guarantee that a rollout player that is trained using TD-Gammon for both players will do better than TD-Gammon would against a non-TD-Gammon opponent. While this is a plausible practical hypothesis, it is one that can only be tested empirically. In fact relevant counterexamples have been constructed for the game of Go using "adversarial" optimization techniques; see Wang et al. [WGB22], and also our discussion on minimax problems in Section 2.12.

Most of the currently existing computer backgammon programs descend from TD-Gammon. Rollout-based backgammon programs are the most powerful in terms of performance, consistent with the principle that a rollout algorithm performs better than its base heuristic. However, they are too time-consuming for real-time play (without parallel computing hardware), because of the extensive on-line simulation requirement at each move.† They have been used in a limited diagnostic way to assess the quality of neural network-based programs (many articles and empirical works on computer backgammon are posted on-line; see e.g., http://www.bkgm.com/articles/page07.html).

2.7.4 Variance Reduction in Rollout - Comparing Advantages

When using simulation, sampling is often organized to effect *variance reduction*. By this we mean that for a given problem, the collection and use of samples is structured so that the variance of the simulation error is made smaller, with the same amount of simulation effort. There are several methods of this type for which we refer to textbooks on simulation (see, e.g., Ross [Ros12], and Rubinstein and Kroese [RuK1]).

† The situation in backgammon is exacerbated by its high branching factor, i.e., for a given position, the number of possible successor positions is quite large, as compared for example with chess.

In this section we discuss a method to reduce the effects of the simulation error in the calculation of the Q-factors in the context of rollout. The key idea is that the selection of the rollout control depends on the values of the Q-factor differences

$$\tilde{Q}_{k,\pi}(x_k, u_k) - \tilde{Q}_{k,\pi}(x_k, \hat{u}_k)$$

for all pairs of controls $(u_k, \hat{u}_k)$. These values must be computed accurately, so that the controls u_k and $\hat{u}_k$ can be accurately compared. On the other hand, the simulation/approximation errors in the computation of the individual Q-factors $\tilde{Q}_{k,\pi}(x_k, u_k)$ may be magnified through the preceding differencing operation.

An approach to counteract this type of simulation error magnification is to approximate the Q-factor difference $\tilde{Q}_{k,\pi}(x_k, u_k) - \tilde{Q}_{k,\pi}(x_k, \hat{u}_k)$ by sampling the difference

$$C_k(x_k, u_k, \mathbf{w}_k) - C_k(x_k, \hat{u}_k, \mathbf{w}_k), \tag{2.78}$$

where $\mathbf{w}_k = (w_k, w_{k+1}, \ldots, w_{N-1})$ is the *same disturbance sequence* for the two controls u_k and $\hat{u}_k$, and

$$C_k(x_k, u_k, \mathbf{w}_k) = g_N(x_N) + g_k(x_k, u_k, w_k) + \sum_{i=k+1}^{N-1} g_i\big(x_i, \mu_i(x_i), w_i\big),$$

with $\{\mu_{k+1}, \ldots, \mu_{N-1}\}$ being the tail portion of the base policy. †

For a simple example that illustrates how this form of variance reduction works, suppose we want to calculate the difference $q_1 - q_2$ of two numbers q_1 and q_2 by subtracting two simulation samples $s_1 = q_1 + w_1$ and $s_2 = q_2 + w_2$, where w_1 and w_2 are zero mean random variables. Then $s_1 - s_2$ is unbiased in the sense that its mean is equal to $q_1 - q_2$. However, the variance of $s_1 - s_2$ decreases as the correlation of w_1 and w_2 increases. It is maximized when w_1 and w_2 are uncorrelated, and it is minimized (it is equal to 0) when w_1 and w_2 are equal.

The preceding example suggests a simulation scheme that is based on the difference (2.78) and involves a common disturbance $\mathbf{w}_k$ for u_k and $\hat{u}_k$. In particular, it may be far more accurate than the one obtained by differencing samples of $C_k(x_k, u_k, \mathbf{w}_k)$ and $C_k(x_k, \hat{u}_k, \hat{\mathbf{w}}_k)$, which involve two different disturbances $\mathbf{w}_k$ and $\hat{\mathbf{w}}_k$. Indeed, by introducing the zero mean sample errors

$$D_k(x_k, u_k, \mathbf{w}_k) = C_k(x_k, u_k, \mathbf{w}_k) - \tilde{Q}_{k,\pi}(x_k, u_k),$$

† For this to be possible, we need to assume that the probability distribution of each disturbance w_i does not depend on x_i and u_i.

it can be seen that the variance of the error in estimating $\tilde{Q}_{k,\pi}(x_k, u_k) - \tilde{Q}_{k,\pi}(x_k, \hat{u}_k)$ with the former method will be no larger than with the latter method if and only if

$$E_{\mathbf{w}_k, \hat{\mathbf{w}}_k}\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k) - D_k(x_k, \hat{u}_k, \hat{\mathbf{w}}_k)\big|^2\Big\} \\ \geq E_{\mathbf{w_k}}\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k) - D_k(x_k, \hat{u}_k, \mathbf{w}_k)\big|^2\Big\}.$$

By expanding the quadratic forms and using the fact $E\big\{D_k(x_k, u_k, \mathbf{w}_k)\big\} = 0$, we see that this condition is equivalent to

$$E\big\{D_k(x_k, u_k, \mathbf{w}_k)D_k(x_k, \hat{u}_k, \mathbf{w}_k)\big\} \geq 0; \tag{2.79}$$

i.e., the errors $D_k(x_k, u_k, \mathbf{w}_k)$ and $D_k(x_k, \hat{u}_k, \mathbf{w}_k)$ being nonnegatively correlated. A little thought should convince the reader that this property is likely to hold in many types of problems.

Roughly speaking, the relation (2.79) holds if changes in the value of u_k (at the first stage) have little effect on the value of the error $D_k(x_k, u_k, \mathbf{w}_k)$ relative to the effect induced by the randomness of $\mathbf{w}_k$. To see this, suppose that there exists a scalar $\gamma < 1$ such that, for all x_k, u_k, and $\hat{u}_k$, there holds

$$E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k) - D_k(x_k, \hat{u}_k, \mathbf{w}_k)\big|^2\Big\} \leq \gamma E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\}. \tag{2.80}$$

Then we have, by using the generic relation $ab \geq a^2 - |a| \cdot |b - a|$ for two scalars a and b,

$$\begin{aligned} &D_k(x_k, u_k, \mathbf{w}_k)D_k(x_k, \hat{u}_k, \mathbf{w}_k) \\ &\qquad \geq \big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2 \\ &\qquad\quad - \big|D_k(x_k, u_k, \mathbf{w}_k)\big| \cdot \big|D_k(x_k, \hat{u}_k, \mathbf{w}_k) - D_k(x_k, u_k, \mathbf{w}_k)\big|, \end{aligned}$$

from which we obtain

$$\begin{aligned} &E\big\{D_k(x_k,u_k, \mathbf{w}_k)D_k(x_k, \hat{u}_k, \mathbf{w}_k)\big\} \\ &\qquad \geq E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\} \\ &\qquad\quad - E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big| \cdot \big|D_k(x_k, \hat{u}_k, \mathbf{w}_k) - D_k(x_k, u_k, \mathbf{w}_k)\big|\Big\} \\ &\qquad \geq E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\} - \frac{1}{2}E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\} \\ &\qquad\quad - \frac{1}{2}E\Big\{\big|D_k(x_k, \hat{u}_k, \mathbf{w}_k) - D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\} \\ &\qquad \geq \frac{1-\gamma}{2}E\Big\{\big|D_k(x_k, u_k, \mathbf{w}_k)\big|^2\Big\}, \end{aligned}$$

where for the second inequality we use the generic relation

$$-|a| \cdot |b| \geq -\frac{1}{2}(a^2 + b^2)$$

for two scalars a and b, and for the third inequality we use Eq. (2.80).

Thus, under the assumption (2.80), the condition (2.79) holds and guarantees that by averaging cost difference samples rather than differencing (independently obtained) averages of cost samples, the simulation error variance does not increase.

Let us finally note the potential benefit of using Q-factor differences in contexts other than rollout. In particular when approximating Q-factors $Q_{k,\pi}(x_k, u_k)$ using parametric architectures (Section 3.3 in Chapter 3), it may be important to approximate and compare instead the differences

$$A_{k,\pi}(x_k, u_k) = Q_{k,\pi}(x_k, u_k) - \min_{u_k \in U_k(x_k)} Q_{k,\pi}(x_k, u_k).$$

The function $A_{k,\pi}(x_k, u_k)$ is also known as the *advantage of the pair* (x_k, u_k), and can serve just as well as $Q_{k,\pi}(x_k, u_k)$ for the purpose of comparing controls, but may work better in the presence of approximation errors. The use of advantages will be discussed further in Chapter 3.

2.7.5 Monte Carlo Tree Search

In our earlier discussion of simulation-based rollout implementation, we implicitly assumed that once we reach state x_k, we generate the same large number of trajectories starting from each pair (x_k, u_k), with $u_k \in U(x_k)$, to the end of the horizon. The drawback of this is threefold:

(a) The trajectories may be too long because the horizon length N is large (or infinite, in an infinite horizon context).

(b) Some of the controls u_k may be clearly inferior to others, and may not be worth as much sampling effort.

(c) Some of the controls u_k that appear to be promising, may be worth exploring better through multistep lookahead.

This has motivated multistep lookahead variants, generally referred to as *Monte Carlo tree search* (MCTS for short), which aim to trade off computational economy with a hopefully small risk of degradation in performance. Such variants involve, among others, early discarding of controls deemed to be inferior based on the results of preliminary calculations, and simulation that is limited in scope (either because of a reduced number of simulation samples, or because of a shortened horizon of simulation, or both).

A simple remedy for (a) above is to use rollout trajectories of reasonably limited length, with some terminal cost approximation at the end (in

an extreme case, the rollout may be skipped altogether for some states, i.e., rollout trajectories have zero length). The terminal cost function may be very simple (such as zero) or may be obtained through some auxiliary calculation. In fact the base policy used for rollout may be used to construct the terminal cost function approximation, as noted for the rollout-based backgammon algorithm of Example 2.7.4. In particular, an approximation to the cost function of the base policy may be obtained by training some approximation architecture, such as a neural network (see Chapter 3), and may be used as a terminal cost function.

A simple but less straightforward remedy for (b) is to use some heuristic or statistical test to discard some of the controls u_k, as soon as this is suggested by the early results of simulation, or even before any simulation, based for example on the suggestions of a trained neural network. Similarly, to implement (c) one may use some heuristic to increase the length of lookahead selectively for some of the controls u_k. This is similar to the incremental multistep rollout scheme for deterministic problems that we discussed in Section 2.4.2; see Fig. 2.4.6.

The MCTS approach can be based on sophisticated procedures for implementing and combining the remedies just described. The general idea is to use the interim results of the computation and statistical tests to focus the simulation effort along the most promising directions. Thus to implement MCTS with multistep lookahead, one needs to maintain a lookahead tree, which is expanded as the relevant Q-factors are evaluated by simulation, and which balances *the competing desires of exploitation and exploration* (generate and evaluate controls that seem most promising in terms of performance versus assessing the potential of inadequately explored controls). Ideas that were developed in the context of multiarmed bandit problems have played an important role in the construction of this type of MCTS procedures (see the end-of-chapter references).

In the simple case of one-step lookahead, with Q-factors calculated by Monte Carlo simulation, MCTS fundamentally aims to find efficiently the minimum of the expected values of a finite number of random variables. This is illustrated in the following example.

Example 2.7.5 (Statistical Tests for Adaptive Sampling with One-Step Lookahead)

Let us consider a typical one-step lookahead selection strategy that is based on adaptive sampling. We are at a state x_k and we try to find a control $\tilde{u}_k$ that minimizes an approximate Q-factor

$$\tilde{Q}_k(x_k, u_k) = E\Big\{g_k(x_k, u_k, w_k) + \tilde{J}_{k+1}\big(f_k(x_k, u_k, w_k)\big)\Big\}$$

over $u_k \in U_k(x_k)$, with $\tilde{Q}_k(x_k, u_k)$ computed by averaging samples of the expression within braces. We assume that $U_k(x_k)$ contains m elements, which

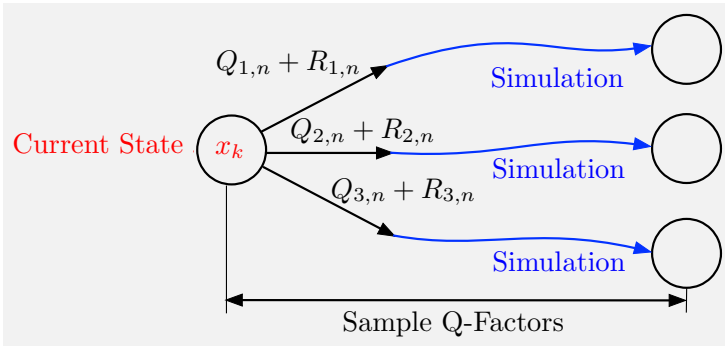

Figure 2.7.3 Illustration of one-step lookahead MCTS at a state x_k. The Q-factor sampled next corresponds to the control i with minimum sum of exploitation index (here taken to be the running average $Q_{i,n}$) and exploration index ($R_{i,n}$, possibly given by the UCB rule).

for simplicity are denoted $1, \ldots, m$. At the ℓth sampling period, knowing the outcomes of the preceding sampling periods, we select one of the m controls, say i_ℓ, and we draw a sample of $\tilde{Q}_k(x_k, i_\ell)$, whose value is denoted by S_{i_ℓ}. Thus after the nth sampling period we have an estimate $Q_{i,n}$ of the Q-factor of each control $i = 1, \ldots, m$ that has been sampled at least once, given by

$$Q_{i,n} = \frac{\sum_{\ell=1}^{n} \delta(i_\ell = i) S_{i_\ell}}{\sum_{\ell=1}^{n} \delta(i_\ell = i)},$$

where

$$\delta(i_\ell = i) = \begin{cases} 1 & \text{if } i_\ell = i, \\ 0 & \text{if } i_\ell \neq i. \end{cases}$$

Thus $Q_{i,n}$ is the *empirical mean* of the Q-factor of control i (total sample value divided by total number of samples), assuming that i has been sampled at least once.

After n samples have been collected, with each control sampled at least once, we may declare the control i that minimizes $Q_{i,n}$ as the "best" one, i.e., the one that truly minimizes the Q-factor $Q_k(x_k, i)$. However, there is a positive probability that there is an error: the selected control may not minimize the true Q-factor. In adaptive sampling, roughly speaking, we want to design the sample selection strategy and the criterion to stop the sampling, in a way that keeps the probability of error small (by allocating some sampling effort to all controls), and the number of samples limited (by not wasting samples on controls i that appear inferior based on their empirical mean $Q_{i,n}$).

Intuitively, a good sampling policy will balance at time n the desires of exploitation and exploration (i.e., sampling controls that seem most promising, in the sense that they have a small empirical mean $Q_{i,n}$, versus assessing the potential of inadequately explored controls, those i that have been sampled a small number of times). Thus it makes sense to sample next the control i that minimizes the sum

$$T_{i,n} + R_{i,n}$$

of two indexes: an *exploitation index* $T_{i,n}$ and an *exploration index* $R_{i,n}$. Usually the exploitation index is chosen to be the empirical mean $Q_{i,n}$; see Fig. 2.7.3. The exploration index is based on a confidence interval formula and depends on the sample count

$$s_i = \sum_{\ell=1}^{n} \delta(i_\ell = i)$$

of control i. A frequently suggested choice is the UCB rule (upper confidence bound), which sets

$$R_{i,n} = -c\sqrt{\frac{\log n}{s_i}},$$

where c is a positive constant that is selected empirically (some analysis suggests values near $c = \sqrt{2}$, assuming that $Q_{i,n}$ is normalized to take values in the range $[-1, 0]$). The UCB rule, first proposed in the paper by Auer, Cesa-Bianchi, and Fischer [ACF02], has been extensively discussed in the literature both for one-step and for multistep lookahead [where it is called UCT (UCB applied to trees; see Kocsis and Szepesvari [KoS06])].†

Its justification is based on probabilistic analyses that relate to the multiarmed bandit problem, and is beyond our scope. Alternatives to the UCB formula have been suggested, and in fact in the AlphaZero program, the exploitation term has a different form than the one above, and depends on the depth of lookahead (see Silver et al. [SHS17]).

Sampling policies for MCTS with multistep lookahead are based on similar sampling ideas to the case of one-step lookahead. A simulated trajectory is run from a node i of the lookahead tree that minimizes the sum $T_{i,n} + R_{i,n}$ of an exploitation index and an exploration index. There are several schemes of this type, but the details are beyond our scope and are often problem-dependent (see the end-of-chapter references).

A major success has been the use of MCTS in two-player game contexts, such as the AlphaGo program (Silver et al. [SHM16]), which performs better than the best humans in the game of Go. This program integrates several of the techniques discussed in this book, including MCTS and rollout using a base policy that is trained off-line using a deep neural network. The AlphaZero program, which has performed spectacularly well against humans and other programs in the games of Go and chess (Silver et al. [SHS17]), bears some similarity with AlphaGo, and critically relies on MCTS, but does not use rollout in its on-line playing mode (it relies primarily on very long lookahead). Let us also note that based on early tests, MCTS is not effective in computer backgammon and it has not been adopted in commercial programs. More generally, the success of MCTS appears to be problem-dependent as well as implementation-dependent.

† The paper [ACF02] refers to the rule given here as UCB1 and credits its motivation to the paper by Agrawal [Agr95]. The book by Lattimore and Szepesvari [LaS20] provides an extensive discussion of the UCB rule and its generalizations.

2.7.6 Randomized Policy Improvement by Monte Carlo Tree Search

We have described rollout and MCTS as schemes for policy improvement: start with a base policy, and compute an improved policy based on the results of one-step lookahead or multistep lookahead followed by simulation with the base policy. We have implicitly assumed that both the base policy and the rollout policy are deterministic in the sense that they map each state x_k into a unique control $\tilde{\mu}_k(x_k)$ [cf. Eq. (2.77)]. In some (even nonstochastic) contexts, success has been achieved with *randomized policies*, which map a state x_k to a probability distribution over the set of controls $U_k(x_k)$, rather than mapping onto a single control. In particular, the AlphaGo and AlphaZero programs use MCTS to generate and use for training purposes randomized policies, which specify at each board position the probabilities with which the various moves are selected.

A randomized policy can be used as a base policy in a rollout context in exactly the same way as a deterministic policy: for a given state x_k, we just generate sample trajectories and associated sample Q-factors, using probabilistically selected controls, starting from each leaf-state of the lookahead tree that is rooted at x_k. We then average the corresponding Q-factor samples. The rollout/improved policy, as described here, is a deterministic policy, i.e., it applies at x_k the control $\tilde{\mu}_k(x_k)$ that is "best" according to the results of the rollout [cf. Eq. (2.77)]. Still, however, if we wish to generate an improved policy that is randomized, we can simply change the probabilities of different controls in the direction of the deterministic rollout policy. This can be done by increasing by some amount the probability of the "best" control $\tilde{\mu}_k(x_k)$ from its base policy level, while proportionally decreasing the probabilities of the other controls.

The use of MCTS provides a related method to "improve" a randomized policy. In the process of the adaptive simulation that is used in MCTS, we generate *frequency counts* of the different controls in $U_k(x_k)$, i.e., the proportion of rollout trajectories associated with each $u_k \in U_k(x_k)$. We can then obtain the rollout randomized policy by moving the probabilities of the base policy in the direction suggested by the frequency counts, i.e., increase the probability of high-count controls and reduce the probability of the others. This type of policy improvement is reminiscent of gradient-type methods, and has been successful in some contexts; see the end-of-chapter references for such policy improvement implementations in AlphaGo, AlphaZero, and other applications.

2.8 ROLLOUT FOR INFINITE-SPACES PROBLEMS - OPTIMIZATION HEURISTICS

We have considered so far finite control space applications of rollout, so there is a finite number of relevant Q-factors at each state x_k, which are

evaluated by simulation and are exhaustively compared. When the control constraint set is infinite, to implement this approach the constraint set must be replaced by a finite set, obtained by some form of discretization or random sampling, which can be inconvenient and ineffective. In this section we will discuss an alternative approach to deal with an infinite number of controls and Q-factors at x_k. The idea is to *use a base heuristic that involves a continuous optimization*, and to rely on a linear or nonlinear programming method to solve the corresponding lookahead optimization problem.

2.8.1 Rollout for Infinite-Spaces Deterministic Problems

To develop the basic idea of how to deal with infinite control spaces, we first consider deterministic problems, involving a system $x_{k+1} = f_k(x_k, u_k)$, and a cost per stage $g_k(x_k, u_k)$. The rollout minimization is

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \tilde{Q}_k(x_k, u_k), \tag{2.81}$$

where $\tilde{Q}_k(x_k, u_k)$ is the approximate Q-factor

$$\tilde{Q}_k(x_k, u_k) = g_k(x_k, u_k) + H_{k+1}\big(f_k(x_k, u_k)\big), \tag{2.82}$$

with $H_{k+1}(x_{k+1})$ being the cost of the base heuristic starting from state x_{k+1} [cf. Eq. (2.10)]. Suppose that we have a differentiable closed-form expression for H_{k+1}, and the functions g_k and f_k are known and are differentiable with respect to u_k. Then the Q-factor $\tilde{Q}_k(x_k, u_k)$ of Eq. (2.82) is also differentiable with respect to u_k, and its minimization (2.81) may be addressed with one of the many gradient-based methods that are available for differentiable unconstrained and constrained optimization.

The preceding approach requires that the heuristic cost $H_{k+1}(x_{k+1})$ can be differentiated, so it should either be available in closed form, which is quite restrictive, or that it can be differentiated numerically, which may be inconvenient and/or unreliable. These difficulties can be circumvented by *using a base heuristic that is itself based on multistep optimization*. In particular, suppose that $H_{k+1}(x_{k+1})$ is the optimal cost of some $(\ell - 1)$-stage deterministic optimal control problem that is related to the original problem. Then the rollout algorithm (2.81)-(2.82) can be implemented by solving the *ℓ-stage deterministic optimal control problem, which seamlessly concatenates the first stage minimization over u_k [cf. Eq. (2.81)], with the $(\ell-1)$-stage minimization of the base heuristic*; see Fig. 2.8.1. This ℓ-stage problem may be solvable on-line by standard continuous spaces nonlinear

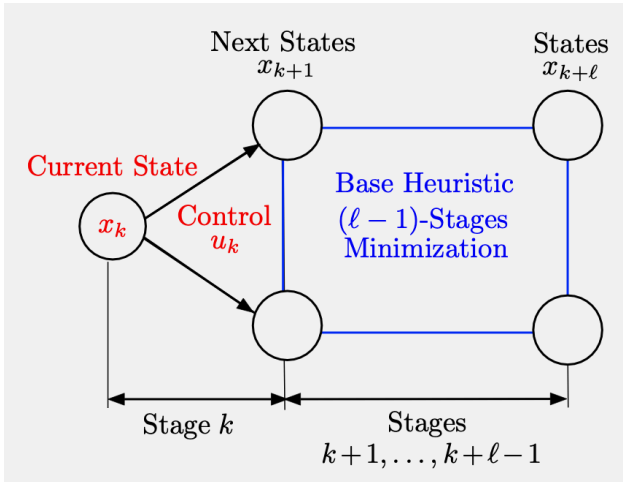

Figure 2.8.1 Schematic illustration of rollout for a deterministic problem with infinite control spaces. The base heuristic is to solve an $(\ell-1)$-stage deterministic optimal control problem, which together with the kth stage minimization over $u_k \in U_k(x_k)$, seamlessly forms an ℓ-stage continuous spaces optimal control/nonlinear programming problem that starts at state x_k.

programming or optimal control methods.† A major paradigm of methods of this type is model predictive control, which we have discussed in Chapter 1 (cf. Section 1.6.9). In the present section we will discuss a few other possibilities. The following is a simple example of an important class of inventory storage and supply chain management processes.

Example 2.8.1 (Supply Chain Management)

Let us consider a supply chain system, where a certain item is produced at a production center and fulfilled at a retail center. Stock of the item is shipped from the production center to the retail center, where it arrives with a delay of $\tau \geq 1$ time units, and is used to fulfill a known stream of demands d_k over an N-stage horizon; see Fig. 2.8.2. We denote:

x_k^1: The stock at hand at the production center at time k.

x_k^2: The stock at hand at the retail center at time k, and used to fulfill demand (both positive and negative x_k^2 are allowed; a negative value indicates that there is backordered demand).

† Note, however, that for this to be possible, it is necessary to have a mathematical model of the system; a simulator is not sufficient. Another difficulty occurs when the control space is the union of a discrete set and a continuous set. Then it may be necessary to use some type of mixed integer programming technique to solve the ℓ-stage problem. Alternatively, it may be possible to handle the discrete part by brute force enumeration, followed by continuous optimization.